



SC1006

Computer

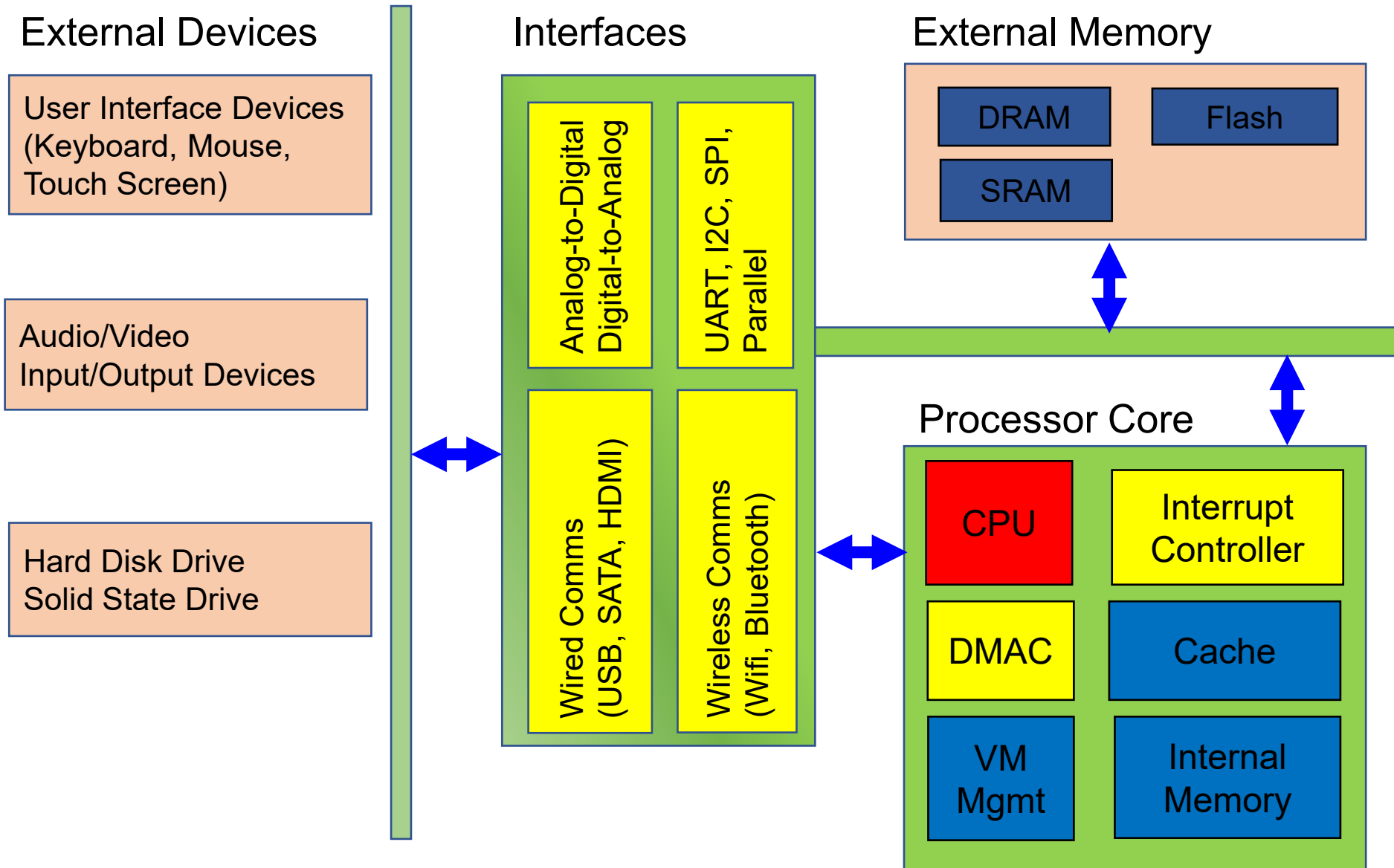
Organisation and

Architecture

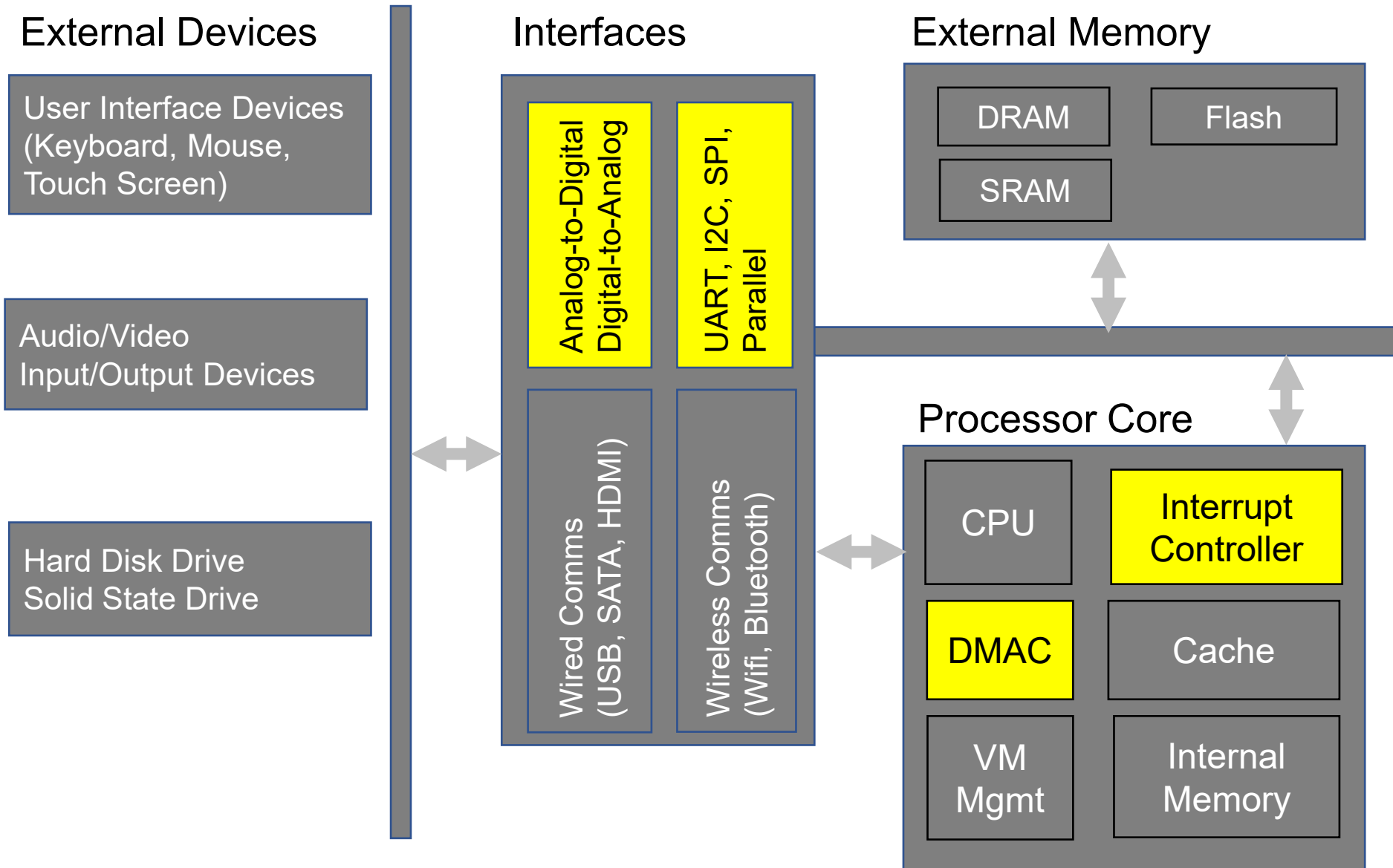
Data Transfer Mechanism

Prof Lin WeiSi
College of Computing and Data Science, NTU
Email: wslin@ntu.edu.sg

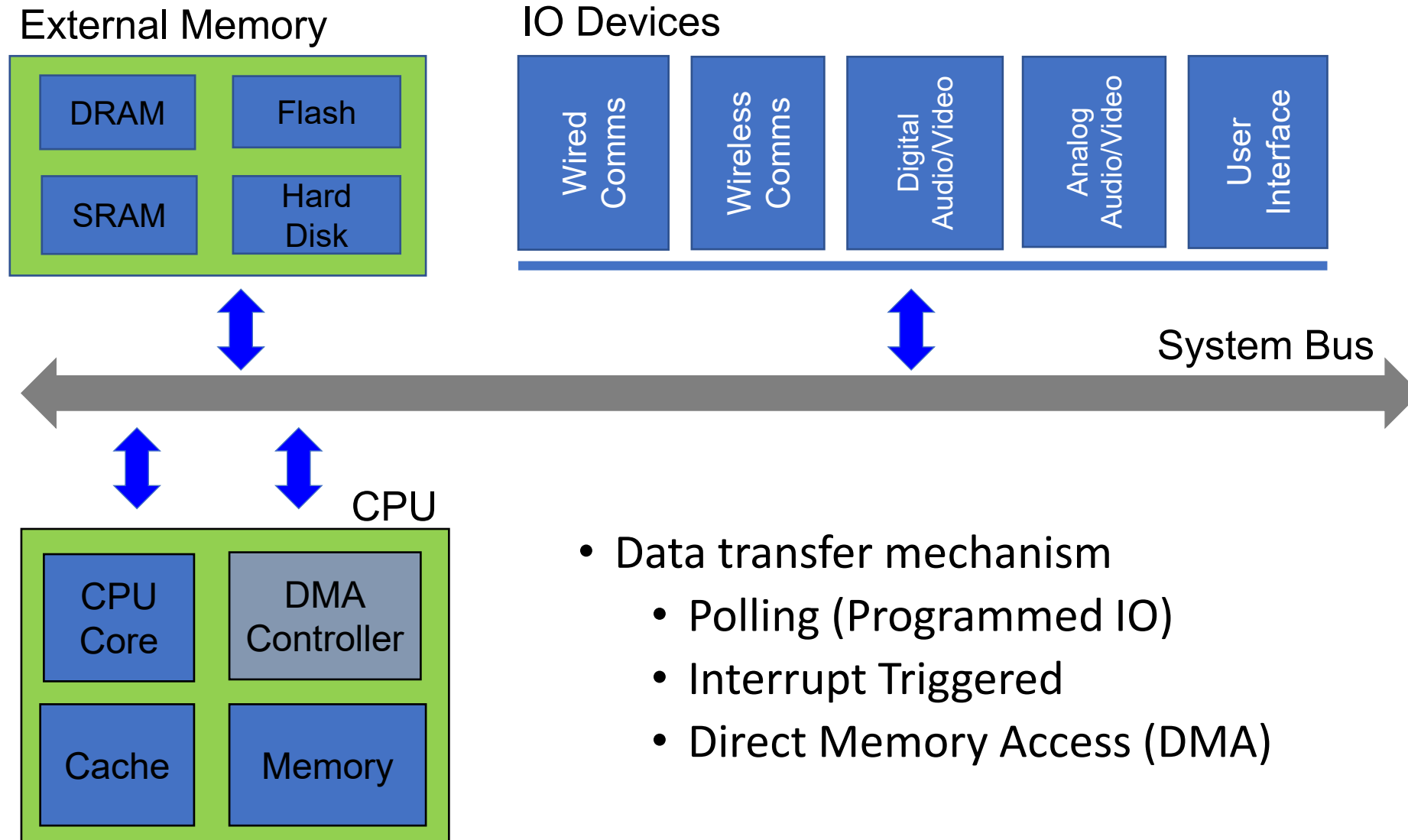
Computer System Block Diagram



Computer System Block Diagram



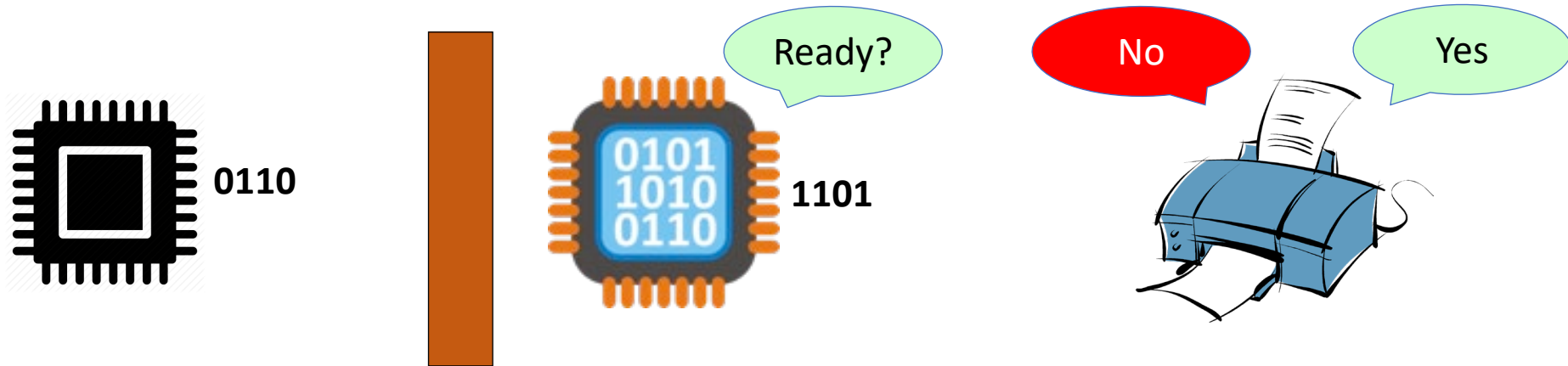
Data Transfer Mechanism



- Data transfer mechanism
 - Polling (Programmed IO)
 - Interrupt Triggered
 - Direct Memory Access (DMA)

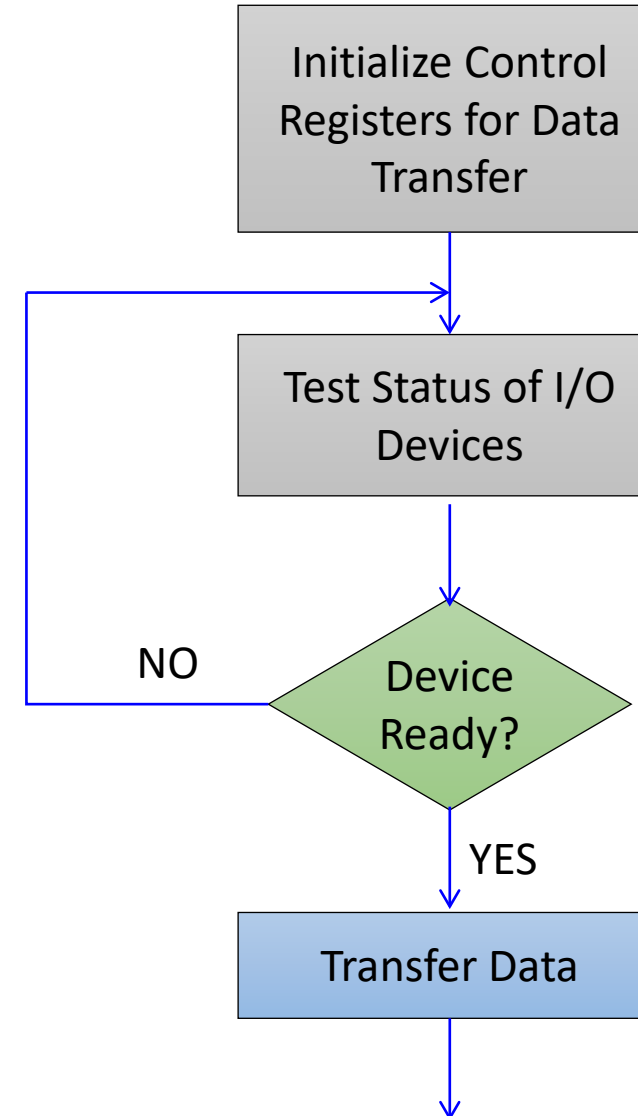
Polling Technique

- CPU polls a certain I/O port **continuously** (using **software**) for **data** or **readiness** of the port to perform a data transaction.
 - For example, the CPU polls the printer port continuously to see if the printer is ready to accept data.
 - If it is ready, the CPU writes a data byte to the printer port. Else, it waits.
- CPU has **full control** and **dedicate 100%** of its resource in the whole data transfer process and does nothing else.



Polling Technique - Flowchart

- CPU performs all necessary initialization.
- CPU polls the I/O device for its readiness to perform data transfer.
- If I/O device is **not ready**, CPU continue to **wait in the loop** to check if device is ready.
- If device is **ready**, CPU makes the data transfer and **exit the loop**.



Pros/Cons of Polling Technique

- Advantages

- Programmer has complete control over the entire process.
- Easiest method to test and debug.

- Disadvantages

- Since the CPU waits in a loop, it cannot perform any other task until data transfer is completed.
- Program execution of CPU held up while waiting for I/O device to get ready.
- Inefficient use of CPU resources.

Interrupts

- A **signalling mechanism** that allows internal and external peripherals to **alert** the CPU that **attention** is **needed**.
- Could be in the form of an external/internal **electrical pulse** or **change in internal register status**.
- Once the CPU receive the signal, known as **interrupt request**, it would then need to decide if it wanted to service the request.
- If CPU decides to service interrupt request, it will follow up with the series of procedures to handle the interrupt event.
- Interrupt mechanism is **typically used trigger the CPU to start some operation**, e.g. data transfer to memory, status registers, control registers etc.

Polling vs Interrupt

- Boiling Water Analogy

Polling:

Check every few minutes



Interrupt:

Listen for the whistle



Interrupt Triggered Data Transfer

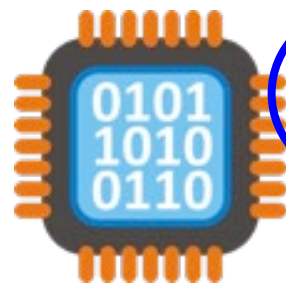
Main Routine

```
main MOV R0, #0
      MOV R2, #8

Loop  ADD R1,R1,R2
      BEQ skip
      ADD R0,R0,R2

skip  .....
```

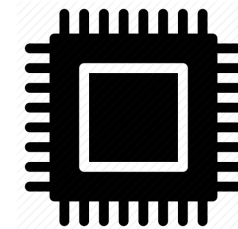
I/O Device #3
Interrupt Service
Routine



INTERRUPT REQUEST

DATA TRANSFER

I/O Device #3



interrupt latency: delay between

- the point where CPU receives the interrupt request
- To the point where it enter the ISR (Interrupt Service Routine)

Interrupt Vector Table

Interrupt Vector Table

0	Reset ISR Addr
1	...
2	...
3	...
4	...
5	...
6	...
7	Ext INT 3 ISR Addr
8	SPI ISR Addr
9	UART ISR Addr
A	

- How does the CPU know the **location** of the corresponding interrupt service routine for each interrupts?
- The **starting address of each interrupt** is stored in a table known as the **interrupt vector table**
- Each interrupt has a **unique index to the vector table**, e.g. UART interrupt could be in index 9, so if a UART interrupt occurs, CPU will know where to branch to by checking index 9 in the vector table.
- The values and interrupt source in the table on the left are for **illustration purpose only**, different processor has different indexing for their interrupts.

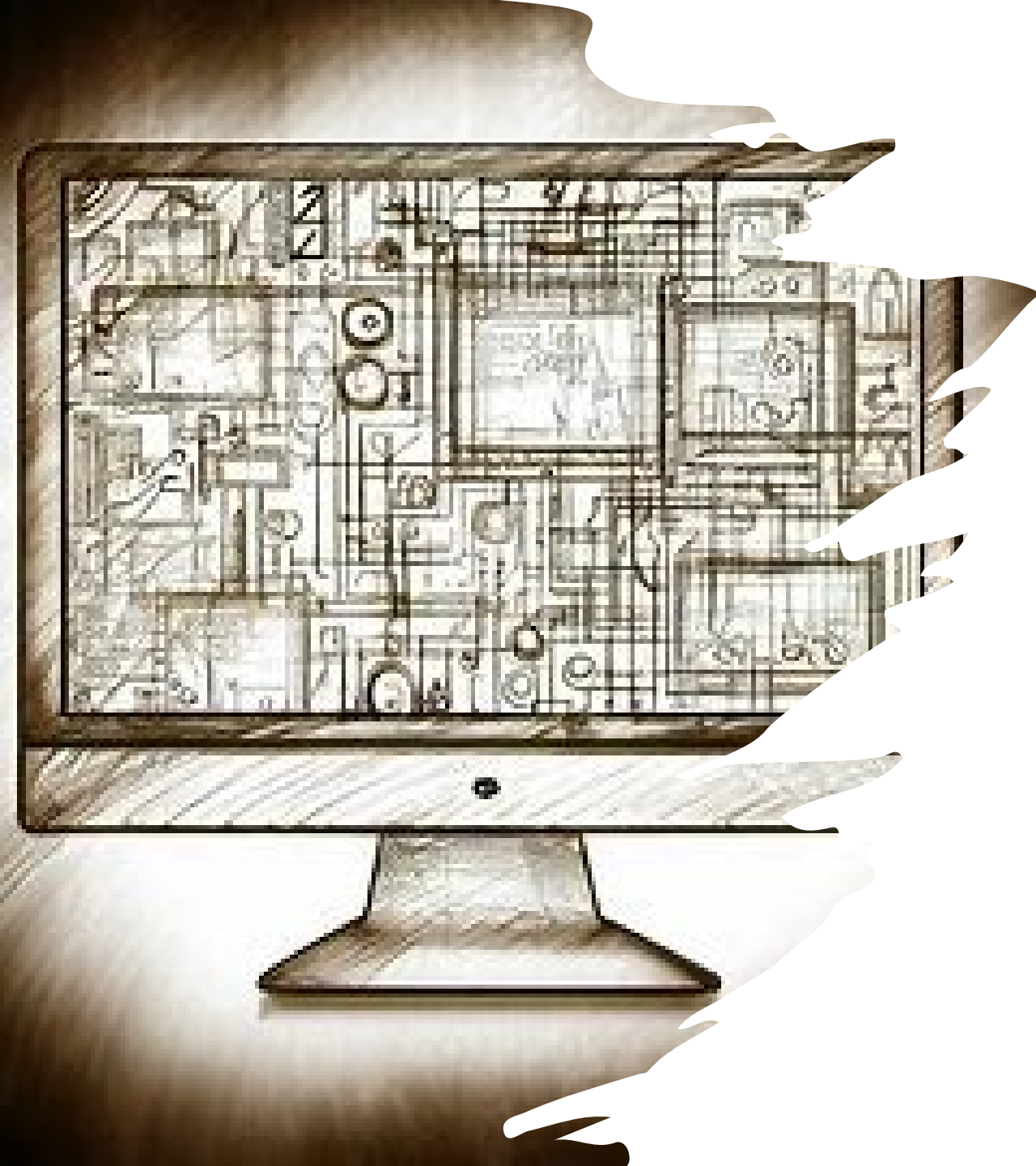
Interrupt Control Flow

- A signal from external/internal peripheral, or a change in status of some special registers notifies the CPU that some event has occur and ask for CPU's (immediate) attention.
- If the CPU decide to service the interrupt, it will suspend its current program temporarily.
- CPU look up the interrupt vector table to check the starting address of the interrupt service routine (ISR) for the corresponding interrupt.
- CPU would save a copy of the processor context, i.e. the current value of various registers, this is to allow the interrupted routine to continue its execution after returning from the ISR.
- CPU then proceed to execute the ISR linked to the interrupt that was triggered.
- Once the ISR is completed, CPU restore the save context, returns to the interrupted routine and continue from where it had left previously.



Interrupt Service Routine (ISR)

- The ISR will perform some operation e.g. Data Transfer.
- ISR is typically a very short routine so as not to suspend the main program for too long.
- After servicing the ISR, CPU will return to the previous program and continue from it branched off.
- It is possible for CPU to receive multiple interrupt requests simultaneously since it typically interface to multiple devices.
- In this case, some arbitration scheme has to be designed to decide which interrupt to service first (priority, first-come-first-serve etc).



Pros/Cons of Interrupt Triggered Technique

- Advantages
 - Efficient use of CPU resources as it does not need to monitor I/O device status.
 - CPU can continue with other tasks between interrupts.
 - Allows prioritization and pre-emption.
- Disadvantages
 - More hardware interface circuitry required between I/O device and processor.
 - Program is slightly more complex and difficult to debug.

